```python
# 1.1 FCFS scheduling algorithm

def calculate_total_time(arrival_times, pages):
 current_time = 0
 total_time = 0

 for i in pages:
  total_time+=i
 return total_time

arrival_times = [0, 1, 3, 5]
pages = [10, 5, 3, 7]
total_time = calculate_total_time(arrival_times, pages)

print(f"The total time taken to complete all printing tasks is: {total_time} seconds")



# 1.2 SJF scheduling algorithm

def calculate_total_time(execution_times):
 total_time = 0

 for i in sorted(execution_times):
  total_time+=i
 return total_time

execution_times = [3, 5, 7, 4]
total_time = calculate_total_time(execution_times)

print(f"The total time required to complete all tasks is: {total_time} hours")



# 1.3 Round Robin scheduling algorithm

from collections import deque
def calculate_total_time(orders, time_quantum):
 total_time = 0

 while any(order > 0 for order in orders):
    for i in range(len(orders)):
      if(orders[i]>time_quantum):
        total_time += time_quantum
        orders[i] -= time_quantum
      else:
        total_time += orders[i]
        orders[i] = 0

 return total_time

orders = [5, 3, 8, 6]
time_quantum = 4
total_time = calculate_total_time(orders, time_quantum)
print(f"The total time required to complete all orders is: {total_time} minutes")



# 1.4 Emergency Room Prioritization

from queue import PriorityQueue
def calculate_total_time(patients):
 total_time = 0
 pq = PriorityQueue()
 for i in patients:
   pq.put((i[0],i[1]))
```

```
 while not pq.empty():
   total_time += pq.get()[1]
   print(total_time)
 return total_time

patients = [(2, 10),(3, 8),(4, 15),(1, 5)]

total_time = calculate_total_time(patients)
print(f"The total time required to treat all patients is: {total_time} minutes")
```

```
# 2.1 Multi-Level FCFS Queue Scheduling algorithm

from queue import Queue
def calculate_total_time(system_processes, user_processes):
 total_time = 0
 for time in system_processes.values():
  total_time += time
 for time in user_processes.values():
  total_time += time

 return total_time

system_processes = {'Process A': 5,'Process B': 3,'Process C': 7,}
user_processes = {'Process D': 4, 'Process E': 2, 'Process F': 6,}
total_time = calculate_total_time (system_processes, user_processes)
print(f"The total time required to complete all processes is: {total_time} units")




# 2.2 Managing Job FCFS Scheduling in a Computing Center

from queue import Queue
def calculate_total_time(high_priority_jobs, low_priority_jobs):
 total_time = 0
 for time in high_priority_jobs.values():
  total_time += time
 for time in low_priority_jobs.values():
  total_time += time

 return total_time

high_priority_jobs = {'Job A': 8, 'Job B': 5, 'Job C': 10,}
low_priority_jobs = {'Job D': 6, 'Job E': 3, 'Job F': 7,}
total_time = calculate_total_time(high_priority_jobs, low_priority_jobs)
print(f"The total time required to complete all jobs is: {total_time} units")




# 2.3 Managing Print Jobs in a Shared Printing Environment

from queue import Queue
def calculate_total_time(high_priority_jobs, low_priority_jobs):
 total_time = 0
 for time in high_priority_jobs.values():
  total_time += time
 for time in low_priority_jobs.values():
  total_time += time

 return total_time

high_priority_jobs = {'Job A': 15, 'Job B': 10, 'Job C': 20,}
low_priority_jobs = {'Job D': 5, 'Job E': 8, 'Job F': 3,}
total_time = calculate_total_time(high_priority_jobs, low_priority_jobs)
print(f"The total time required to complete all print jobs is: {total_time} units")



# 2.4 Task Scheduling in a Multi-User System

from queue import Queue
def calculate_total_time(high_priority_tasks, low_priority_tasks):
 total_time = 0
 for time in high_priority_tasks.values():
  total_time += time
 for time in low_priority_tasks.values():
```

```
  total_time += time

 return total_time

high_priority_tasks = {'Task A': 12, 'Task B': 8, 'Task C': 15,}
low_priority_tasks = {'Task D': 6, 'Task E': 4, 'Task F': 10, }
total_time = calculate_total_time(high_priority_tasks, low_priority_tasks)
print(f"The total time required to complete all tasks is: {total_time} units")




# 2.5 Job Scheduling in a Computing Cluster

from queue import Queue
def calculate_total_time(high_priority_jobs, medium_priority_jobs,
low_priority_jobs):
  total_time = 0
  for time in high_priority_jobs.values():
    total_time += time
  for time in medium_priority_jobs.values():
    total_time += time
  for time in low_priority_jobs.values():
    total_time += time

  return total_time

high_priority_jobs = {'Job A': 20, 'Job B': 15, 'Job C': 25,}
medium_priority_jobs = {'Job D': 10, 'Job E': 12, 'Job F': 8,}
low_priority_jobs = {'Job G': 5, 'Job H': 4, 'Job I': 6,}
total_time = calculate_total_time(high_priority_jobs, medium_priority_jobs,
low_priority_jobs)
print(f"The total time required to complete all jobs is: {total_time} units")
```

```python
# 3.1 Managing Student Records in a School Database

class Record:
 def __init__(self, name, student_id, grade, address):
    self.name = name
    self.student_id = student_id
    self.grade = grade
    self.address = address
 def __str__(self):
    return f"Name: {self.name}, ID: {self.student_id}, Grade: {self.grade}, Address:
{self.address}"

class SequentialFileAllocation:

 def __init__(self, block_size):
    self.disk_blocks = []
    self.fat = {}
    self.block_size = block_size
    self.next_free_block = 0

 def add_record(self, student_record):
    return True

 def get_record(self, student_id):

    return None

 def calculate_record_size(self, student_record):

    return 1

 def print_disk_status(self):

   return None


block_size = 1

file_system = SequentialFileAllocation(block_size)
student_records = [Record ("John Doe", 101, 10, "123 Main Street"),Record ("Jane Smith", 102,
11, "456 Elm Street"),Record ("Michael Brown", 103, 9, "789 Oak Avenue")]

for record in student_records:
 file_system.add_record (record)

file_system.print_disk_status()
student_id = 102
retrieved_record = file_system.get_record (student_id)

if retrieved_record:
 print(f"\n Retrieved Record for ID {student_id}:")
 print(retrieved_record)
else:
 print(f"\n Record with ID {student_id} not found.")


# 3.2 ------------------------------------------------------------

class PatientRecord:
    def __init__(self, name, age, medical_id, address):
        self.name = name
        self.age = age
        self.medical_id = medical_id
        self.address = address

    def __str__(self):
        return f"Name: {self.name}, Age: {self.age}, Medical ID: {self.medical_id}, Address:
```

```
{self.address}"


class IndexedFileAllocation:
    def __init__(self, block_size):
        self.disk_blocks = []
        self.index_table = {}
        self.index_block_size = block_size

    def add_record(self, patient_record):
        if len(self.disk_blocks) < self.index_block_size:
            block_index = len(self.disk_blocks)
            self.disk_blocks.append(patient_record)
            self.index_table[patient_record.medical_id] = block_index
            print(f"Added record for {patient_record.name} at block {block_index}.")
            return True
        print("Disk full. Cannot add record.")
        return False

    def get_record(self, medical_id):
        block_index = self.index_table.get(medical_id)
        if block_index is not None:
            return self.disk_blocks[block_index]
        return None

    def print_disk_status(self):
        for i, block in enumerate(self.disk_blocks):
            print(f"Block {i}: {block}")


# Driver Code
index_block_size = 3  # Assuming we can store 3 records
file_system = IndexedFileAllocation(index_block_size)

patient_records = [
    PatientRecord("John Smith", 45, 1001, "123 Hospital Road"),
    PatientRecord("Jane Doe", 32, 1002, "456 Clinic Avenue"),
    PatientRecord("Michael Johnson", 58, 1003, "789 Medical Plaza")
]

for record in patient_records:
    file_system.add_record(record)

file_system.print_disk_status()

medical_id = 1002
retrieved_record = file_system.get_record(medical_id)
if retrieved_record:
    print(f"\nRetrieved Record for Medical ID {medical_id}:")
    print(retrieved_record)
else:
    print(f"\nRecord with Medical ID {medical_id} not found.")


# 3.3 ----------------------------------------------------------------

class DiskBlock:
    def __init__(self, block_id):
        self.block_id = block_id
        self.next_block = None


class MediaFile:
    def __init__(self, file_name, file_type, size):
        self.file_name = file_name
        self.file_type = file_type
        self.size = size
```

```
        self.start_block = None  # Points to the first block in the chain

    def __str__(self):
        return f"{self.file_name} ({self.file_type}), Size: {self.size} MB"


class FileAllocationTable:
    def __init__(self):
        self.fat = {}

    def allocate_blocks(self, media_file, disk_blocks):
        blocks_needed = (media_file.size + 9) // 10  # Assuming 10 MB per block
        for i in range(len(disk_blocks)):
            if disk_blocks[i] is None:  # Found a free block
                if blocks_needed == 0:
                    break
                disk_blocks[i] = DiskBlock(i)
                if media_file.start_block is None:
                    media_file.start_block = disk_blocks[i]
                elif prev_block:
                    prev_block.next_block = disk_blocks[i]
                prev_block = disk_blocks[i]
                blocks_needed -= 1
        if blocks_needed == 0:
            self.fat[media_file.file_name] = media_file.start_block
            print(f"Allocated {media_file} in blocks.")
        else:
            print(f"Not enough blocks to allocate {media_file}.")

    def delete_file(self, media_file, disk_blocks):
        current = media_file.start_block
        while current:
            disk_blocks[current.block_id] = None
            current = current.next_block
        del self.fat[media_file.file_name]
        print(f"Deleted {media_file.file_name}.")


def simulate_linked_file_allocation():
    disk_blocks = [None] * 20  # Simulating 20 blocks of 10 MB each
    fat = FileAllocationTable()

    # Creating media files
    files = [
        MediaFile("Landscape.jpg", "Image", 5),
        MediaFile("Concert.mp4", "Video", 50),
        MediaFile("Song.mp3", "Audio", 8),
    ]

    # Allocating blocks for each file
    for file in files:
        fat.allocate_blocks(file, disk_blocks)

    # Displaying the allocation status
    for i, block in enumerate(disk_blocks):
        print(f"Block {i}: {'Free' if block is None else 'Allocated'}")

    # Deleting a file
    fat.delete_file(files[1], disk_blocks)  # Deleting "Concert.mp4"

    print("\nAfter deleting Concert.mp4:")
    for i, block in enumerate(disk_blocks):
        print(f"Block {i}: {'Free' if block is None else 'Allocated'}")


# Driver Code
simulate_linked_file_allocation()
```

```python
# 3.4 ------------------------------------------------------------

class DigitalArchive:
    def __init__(self, total_blocks, block_size=10):
        self.total_blocks = total_blocks
        self.block_size = block_size
        self.disk_blocks = [None] * total_blocks  # None indicates free block

    def calculate_blocks_needed(self, file_size):
        return (file_size + self.block_size - 1) // self.block_size  # Ceiling division

    def allocate_blocks(self, file_name, file_size):
        blocks_needed = self.calculate_blocks_needed(file_size)
        for i in range(self.total_blocks - blocks_needed + 1):
            if all(self.disk_blocks[j] is None for j in range(i, i + blocks_needed)):
                for j in range(i, i + blocks_needed):
                    self.disk_blocks[j] = file_name
                print(f"Allocated {blocks_needed} blocks for {file_name}.")
                return True
        print(f"Not enough space to allocate {file_name}.")
        return False

    def delete_file(self, file_name):
        for i in range(self.total_blocks):
            if self.disk_blocks[i] == file_name:
                self.disk_blocks[i] = None
        print(f"Deleted {file_name}.")

    def display_allocation_status(self):
        for i, block in enumerate(self.disk_blocks):
            status = block if block else "Free"
            print(f"Block {i}: {status}")


# Driver Code
digital_archive = DigitalArchive(100)

digital_archive.allocate_blocks("Portrait1.jpg", 15)
digital_archive.allocate_blocks("Landscape1.jpg", 10)
digital_archive.allocate_blocks("Architecture1.jpg", 7)

digital_archive.display_allocation_status()

digital_archive.delete_file("Landscape1.jpg")

print("\nAfter deleting Landscape1.jpg:")
digital_archive.display_allocation_status()
```

```python
# 4.1

class MemoryVariableTechnique:
    def __init__(self, partitions):
        self.partitions = partitions
        self.memory_map = {1: [False] * partitions[1],
                           2: [False] * partitions[2],
                           3: [False] * partitions[3]}
        self.processes = {}

    def allocate_memory(self, process_id, size):
        for partition_id, partition_size in self.partitions.items():
            if size <= partition_size:
                start_idx = self.find_free_slot(partition_id, size)
                if start_idx is not None:
                    for i in range(start_idx, start_idx + size):
                        self.memory_map[partition_id][i] = True
                    self.processes[process_id] = (partition_id, size)
                    return
        print(f"Error: Cannot allocate {size} units of memory for process {process_id}")

    def find_free_slot(self, partition_id, size):
        memory_map = self.memory_map[partition_id]
        for i in range(len(memory_map) - size + 1):
            if all(not memory_map[i + j] for j in range(size)):
                return i
        return None

    def deallocate_memory(self, process_id):
        if process_id in self.processes:
            partition_id, size = self.processes[process_id]
            for i in range(size):
                self.memory_map[partition_id][i] = False
            del self.processes[process_id]

    def display_memory_status(self):
        for partition_id, partition_size in self.partitions.items():
            print(f"Partition {partition_id}: ", end="")
            for i in range(partition_size):
                print("X" if self.memory_map[partition_id][i] else "-", end=" ")
            print()

mvt = MemoryVariableTechnique({1: 300, 2: 500, 3: 200})
mvt.allocate_memory(1, 150)
mvt.allocate_memory(2, 400)
mvt.allocate_memory(3, 200)
mvt.display_memory_status()

mvt.deallocate_memory(2)
print("\nAfter deallocating Process 2:")
mvt.display_memory_status()


4## .2 BestFit Memory Manager

class BestFitMemoryManager:
    def __init__(self, memory_blocks):
        self.memory_blocks = memory_blocks
        self.available_blocks = {i: block for i, block in enumerate(memory_blocks)}
        self.processes = {}

    def allocate_memory(self, process_id, size):
        best_fit = None
        for partition_id, partition_size in self.available_blocks.items():
            if partition_size >= size:
                if best_fit is None or partition_size < self.available_blocks[best_fit]:
                    best_fit = partition_id
```

```
        if best_fit is not None:
            self.available_blocks[best_fit] -= size
            self.processes[process_id] = (best_fit, size)
            print(f"Process {process_id} allocated {size} KB in Partition {best_fit}")
            return True
        else:
            print(f"Process {process_id} allocation of {size} KB failed")
            return False

    def deallocate_memory(self, process_id):
        if process_id in self.processes:
            partition_id, size = self.processes.pop(process_id)
            self.available_blocks[partition_id] += size
            print(f"Process {process_id} deallocated from Partition {partition_id}")
        else:
            print(f"Process {process_id} not found")

    def display_memory_status(self):
        print("Current Memory Allocation Status:")
        for partition_id, partition_size in self.available_blocks.items():
            print(f"Partition {partition_id}: {partition_size} KB available")

# Driver Code
memory_manager = BestFitMemoryManager([100, 250, 150])
memory_manager.allocate_memory(1, 70)
memory_manager.allocate_memory(2, 180)
memory_manager.allocate_memory(3, 250)
memory_manager.display_memory_status()
memory_manager.deallocate_memory(2)
print("\nAfter deallocating Process 2:")
memory_manager.display_memory_status()


## 4.3 WorstFit Manager

class WorstFitMemoryManager:
    def __init__(self, memory_blocks):
        self.memory_blocks = memory_blocks
        self.available_blocks = {i: block for i, block in enumerate(memory_blocks)}
        self.processes = {}

    def allocate_memory(self, process_id, size):
        # Find the worst fit block (largest block that can fit the process)
        worst_fit_index = -1
        max_size = -1
        for index, block_size in self.available_blocks.items():
            if block_size >= size and block_size > max_size:
                worst_fit_index = index
                max_size = block_size

        if worst_fit_index == -1:
            print(f"Process {process_id} allocation of {size} KB failed")
            return False

        # Allocate memory
        self.available_blocks[worst_fit_index] -= size
        if self.available_blocks[worst_fit_index] == 0:
            del self.available_blocks[worst_fit_index]
        self.processes[process_id] = (worst_fit_index, size)
        print(f"Process {process_id} allocated {size} KB in Block {worst_fit_index}")
        return True

    def deallocate_memory(self, process_id):
        if process_id in self.processes:
            block_index, size = self.processes.pop(process_id)
            if block_index in self.available_blocks:
```

```
                self.available_blocks[block_index] += size
            else:
                self.available_blocks[block_index] = size
            print(f"Process {process_id} deallocated from Block {block_index}")
        else:
            print(f"Process {process_id} not found")

    def display_memory_status(self):
        print("Current Memory Allocation Status:")
        for index, block_size in self.available_blocks.items():
            print(f"Block {index}: {block_size} KB free")
        for process_id, (block_index, size) in self.processes.items():
            print(f"Process {process_id}: {size} KB in Block {block_index}")

# Driver Code
memory_manager = WorstFitMemoryManager([150, 300,450])
memory_manager.allocate_memory(1, 180)
memory_manager.allocate_memory(2, 270)
memory_manager.display_memory_status()
memory_manager.deallocate_memory(2)
print("\nAfter deallocating Process 2:")
memory_manager.display_memory_status()


# 4.4 MFT Manager

class MFTMemoryManager:
    def __init__(self, num_partitions, partition_size):
        self.num_partitions = num_partitions
        self.partition_size = partition_size
        self.available_partitions = [True] * num_partitions
        self.processes = {}

    def allocate_memory(self, process_id, size):
        if size > self.partition_size:
            print(f"Process {process_id} requires more memory than a single partition can
provide.")
            return False
        for i in range(self.num_partitions):
            if self.available_partitions[i]:
                self.available_partitions[i] = False
                self.processes[process_id] = i
                print(f"Allocated partition {i} to process {process_id}.")
                return True
        print(f"No available partition for process {process_id}.")
        return False

    def deallocate_memory(self, process_id):
        if process_id in self.processes:
            partition_index = self.processes.pop(process_id)
            self.available_partitions[partition_index] = True
            print(f"Deallocated partition {partition_index} from process {process_id}.")
        else:
            print(f"Process {process_id} not found.")

    def display_memory_status(self):
        print("Memory Status:")
        for i in range(self.num_partitions):
            status = "Free" if self.available_partitions[i] else "Occupied"
            print(f"Partition {i}: {status}")

# Driver Code
memory_manager = MFTMemoryManager(num_partitions=8, partition_size=800)
memory_manager.allocate_memory(1, 600)
memory_manager.allocate_memory(2, 900)
memory_manager.allocate_memory(3, 400)
memory_manager.display_memory_status()
```

```
memory_manager.deallocate_memory(2)
print("\nAfter deallocating Process 2:")
memory_manager.display_memory_status()



# 4.5 Simulating Paging Memory Management

class PagingMemoryManager:
    def __init__(self, num_pages, page_size):
        self.num_pages = num_pages
        self.page_size = page_size
        self.available_pages = [True] * num_pages
        self.processes = {}

    def allocate_memory(self, process_id, size):
        num_required_pages = (size + self.page_size - 1) // self.page_size  # Calculate the
number of pages needed
        allocated_pages = []

        for i in range(self.num_pages):
            if len(allocated_pages) == num_required_pages:
                break
            if self.available_pages[i]:
                self.available_pages[i] = False
                allocated_pages.append(i)

        if len(allocated_pages) == num_required_pages:
            self.processes[process_id] = allocated_pages
            print(f"Allocated pages {allocated_pages} to process {process_id}.")
            return True
        else:
            # Rollback allocation if not enough pages are available
            for page in allocated_pages:
                self.available_pages[page] = True
            print(f"Not enough pages available for process {process_id}.")
            return False

    def deallocate_memory(self, process_id):
        if process_id in self.processes:
            allocated_pages = self.processes.pop(process_id)
            for page in allocated_pages:
                self.available_pages[page] = True
            print(f"Deallocated pages {allocated_pages} from process {process_id}.")
        else:
            print(f"Process {process_id} not found.")

    def display_memory_status(self):
        print("Memory Status:")
        for i in range(self.num_pages):
            status = "Free" if self.available_pages[i] else "Occupied"
            print(f"Page {i}: {status}")

# Driver Code
memory_manager = PagingMemoryManager(num_pages=20, page_size=200)
memory_manager.allocate_memory(1, 400)
memory_manager.allocate_memory(2, 600)
memory_manager.allocate_memory(3, 300)
memory_manager.display_memory_status()
memory_manager.deallocate_memory(2)
print("\nAfter deallocating Process 2:")
memory_manager.display_memory_status()
```

```python
# 5.1 Futuristic Space Station

def worst_fit_allocation(memory_blocks,program_requests):
    allocations={}
    remaining_memory = memory_blocks.copy()

    for program, request in program_requests.items():
        largest_block_index = remaining_memory.index(max(remaining_memory))
        largest_block_size = remaining_memory[largest_block_index]

        if largest_block_size >= request:
            allocations[program] = largest_block_size
            remaining_memory[largest_block_index] -= request
        else:
            raise ValueError(f"No memory block can accommodate {program}'s request of
{request} units")

    return allocations, remaining_memory



# Driver Code
memory_blocks = [400, 250, 350, 200, 150]
program_requests = {'Program A': 150, 'Program B': 300, 'Program C': 200}

allocations, remaining_memory = worst_fit_allocation(memory_blocks, program_requests)

# for program, allocated_memory in allocations.items():
#     print(f"{program} allocated {allocated_memory} units of memory.")

print("Remaining Memory in Memory Blocks:")
for i in range(len(memory_blocks)):
    print(f"Block {i+1}: {remaining_memory[i]} units")


#  5.2 Central AI

class MemoryBlock:
    def __init__(self, start, size):
        self.start = start
        self.size = size
        self.allocated = False
        self.process_name = None

    def is_free(self):
        return not self.allocated

    def allocate(self, process_name):
        if self.is_free():
            self.allocated = True
            self.process_name = process_name
            return True
        return False

    def deallocate(self):
        self.allocated = False
        self.process_name = None

    def __str__(self):
        status = "Free" if not self.allocated else f"Allocated to {self.process_name}"
        return f"[Start: {self.start}, Size: {self.size}, Status: {status}]"


class MemoryManager:
    def __init__(self, total_memory):
        self.total_memory = total_memory
        self.memory_blocks = [MemoryBlock(0, total_memory)]
```

```
    def allocate_memory(self, process_name, size):
        for block in self.memory_blocks:
            if block.is_free() and block.size >= size:
                block.allocate(process_name)
                if block.size > size:
                    new_block = MemoryBlock(block.start + size, block.size - size)
                    self.memory_blocks.append(new_block)
                    block.size = size
                return True
        return False

    def print_memory_status(self):
        for block in self.memory_blocks:
            print(block)

    def print_total_memory_used(self):
        total_used = sum(block.size for block in self.memory_blocks if not block.is_free())
        print(f"Total memory used: {total_used} units")


# Driver Code
memory_manager = MemoryManager(8000)

requests = [("A", 1500), ("B", 1000), ("C", 700),("D", 2200), ("E", 500), ("F", 1200)]

for request in requests:
    process_name, size = request
    allocated = memory_manager.allocate_memory(process_name, size)
    if allocated:
        memory_manager.print_memory_status()

memory_manager.print_total_memory_used()


# 5.3 MetroCentral

class MemoryBlock:
    def __init__(self, start, size):
        self.start = start
        self.size = size
        self.allocated = False
        self.process_name = None

    def is_free(self):
        return not self.allocated

    def allocate(self, process_name):
        if self.is_free():
            self.allocated = True
            self.process_name = process_name
            return True
        return False

    def deallocate(self):
        self.allocated = False
        self.process_name = None

    def __str__(self):
        status = "Free" if not self.allocated else f"Allocated to {self.process_name}"
        return f"[Start: {self.start}, Size: {self.size}, Status: {status}]"


class MemoryManager:
    def __init__(self, total_memory):
        self.total_memory = total_memory
        self.memory_blocks = [MemoryBlock(0, total_memory)]  # Start with one large free block
```

```
    def allocate_memory(self, process_name, size):
        for block in self.memory_blocks:
            if block.is_free() and block.size >= size:
                if block.size > size:
                    # Split the block if it's larger than the requested size
                    new_block = MemoryBlock(block.start + size, block.size - size)
                    self.memory_blocks.insert(self.memory_blocks.index(block) + 1, new_block)
                block.size = size
                block.allocate(process_name)
                print(f"Allocated {size} units to {process_name}.")
                return True
        print(f"Failed to allocate {size} units to {process_name}.")
        return False

    def print_memory_status(self):
        print("\nMemory Status:")
        for block in self.memory_blocks:
            print(block)

    def print_total_memory_used(self):
        used_memory = sum(block.size for block in self.memory_blocks if not block.is_free())
        print(f"\nTotal Memory Used: {used_memory}/{self.total_memory} units\n")


# Driver Code
memory_manager = MemoryManager(6000)
requests = [
    ("Project A", 1500),
    ("Project B", 1000),
    ("Project C", 700),
    ("Project D", 2200),
]

for request in requests:
    process_name, size = request
    allocated = memory_manager.allocate_memory(process_name, size)
    if allocated:
        memory_manager.print_memory_status()
        memory_manager.print_total_memory_used()




# 5.4 Ai Lab

class MemoryBlock:
    def __init__(self, start, size):
        self.start = start
        self.size = size
        self.allocated = False
        self.process_name = None

    def is_free(self):
        return not self.allocated

    def allocate(self, process_name):
        if self.is_free():
            self.allocated = True
            self.process_name = process_name
            return True
        return False

    def deallocate(self):
        self.allocated = False
        self.process_name = None
```

```
    def __str__(self):
        status = "Free" if not self.allocated else f"Allocated to {self.process_name}"
        return f"[Start: {self.start}, Size: {self.size}, Status: {status}]"


class MemoryManager:
    def __init__(self, total_memory):
        self.total_memory = total_memory
        self.memory_blocks = [MemoryBlock(0, total_memory)]  # Start with one large free block

    def allocate_memory(self, process_name, size):
        for block in self.memory_blocks:
            if block.is_free() and block.size >= size:
                if block.size > size:
                    # Split the block if it's larger than the requested size
                    new_block = MemoryBlock(block.start + size, block.size - size)
                    self.memory_blocks.insert(self.memory_blocks.index(block) + 1, new_block)
                block.size = size
                block.allocate(process_name)
                print(f"Allocated {size} units to {process_name}.")
                return True
        print(f"Failed to allocate {size} units to {process_name}.")
        return False

    def print_memory_status(self):
        print("\nMemory Status:")
        for block in self.memory_blocks:
            print(block)

    def print_total_memory_used(self):
        used_memory = sum(block.size for block in self.memory_blocks if not block.is_free())
        print(f"\nTotal Memory Used: {used_memory}/{self.total_memory} units\n")


# Driver Code
memory_manager = MemoryManager(8000)
requests = [
    ("Emergency Department", 2000),
    ("Cardiology Department", 1500),
    ("Laboratory Information System", 1200),
    ("Radiology Department", 1800),
    ("Patient Management System", 1000),
    ("Pharmacy System", 600),
    ("Surgical Services", 2200)
]

for request in requests:
    process_name, size = request
    allocated = memory_manager.allocate_memory(process_name, size)
    if allocated:
        memory_manager.print_memory_status()
        memory_manager.print_total_memory_used()


# 5.5 MedTech

class MemoryBlock:
    def __init__(self, start, size):
        self.start = start
        self.size = size
        self.allocated = False
        self.process_name = None

    def is_free(self):
        return not self.allocated
```

```
    def allocate(self, process_name):
        self.allocated = True
        self.process_name = process_name

    def __str__(self):
        return f"{'Free' if not self.allocated else self.process_name} ({self.start},
{self.size})"


class MemoryManager:
    def __init__(self, total_memory):
        self.total_memory = total_memory
        self.memory_blocks = [MemoryBlock(0, total_memory)]

    def allocate_memory(self, process_name, size):
        best_fit_index = -1
        best_fit_size = float('inf')

        # Find the best fit block
        for i, block in enumerate(self.memory_blocks):
            if block.is_free() and block.size >= size and block.size < best_fit_size:
                best_fit_size = block.size
                best_fit_index = i

        if best_fit_index != -1:
            best_fit_block = self.memory_blocks[best_fit_index]
            if best_fit_block.size > size:
                # Split the block if it's larger than the requested size
                new_block = MemoryBlock(best_fit_block.start + size, best_fit_block.size -
size)
                self.memory_blocks.insert(best_fit_index + 1, new_block)
            best_fit_block.size = size
            best_fit_block.allocate(process_name)
            return True
        return False

    def print_memory_status(self):
        for block in self.memory_blocks:
            print(block)

    def print_total_memory_used(self):
        total_used = sum(block.size for block in self.memory_blocks if block.allocated)
        print(f"Total memory used: {total_used} units")


# Driver Code
memory_manager = MemoryManager(8000)

requests = [
    ("Emergency Department", 2000),
    ("Cardiology Department", 1500),
    ("Laboratory Information System", 1200),
    ("Radiology Department", 1800),
    ("Patient Management System", 1000),
    ("Pharmacy System", 600),
    ("Surgical Services", 2200)
]

for request in requests:
    process_name, size = request
    allocated = memory_manager.allocate_memory(process_name, size)
    if allocated:
        print(f"Allocated {size} units for {process_name}")
        memory_manager.print_memory_status()
    else:
        print(f"Not enough memory available for {process_name} (requested {size} units)")
```

```
memory_manager.print_total_memory_used()
```

```python
# 6.1

class Page:
    def __init__(self, page_id, process_name=None):
        """Initialize a page with a unique ID and an optional process name."""
        self.page_id = page_id
        self.process_name = process_name

    def allocate(self, process_name):
        """Allocate the page to the given process."""
        self.process_name = process_name

    def deallocate(self):
        """Deallocate the page, making it free."""
        self.process_name = None

    def __str__(self):
        """Return a string representation of the page."""
        return f"Page {self.page_id}: {self.process_name or 'Free'}"

class MemoryManager:
    def __init__(self, num_pages, page_size):
        """Initialize the memory manager with given number of pages and page size."""
        self.num_pages = num_pages
        self.page_size = page_size
        self.pages = [Page(page_id) for page_id in range(num_pages)]

    def allocate_memory(self, process_name, num_pages_requested):
        """Allocate the requested number of pages to a process."""
        free_pages = [page for page in self.pages if page.process_name is None]

        if len(free_pages) >= num_pages_requested:
            for i in range(num_pages_requested):
                free_pages[i].allocate(process_name)
            return True  # Allocation successful
        else:
            return False  # Not enough free pages

    def print_memory_status(self):
        """Print the status of each page."""
        for page in self.pages:
            print(page)

    def print_total_memory_used(self):
        """Print the total memory used by all allocated pages."""
        used_pages = sum(1 for page in self.pages if page.process_name is not None)
        total_memory_used = used_pages * self.page_size
        print(f"Total memory used: {total_memory_used} KB")

# Driver Code
memory_manager = MemoryManager(num_pages=100, page_size=4)  # 4 KB page size
requests = [
    ("Process A", 25),
    ("Process B", 15),
    ("Process C", 30),
    ("Process D", 12),
    ("Process E", 20)
]

for request in requests:
    process_name, num_pages_requested = request
    allocated = memory_manager.allocate_memory(process_name, num_pages_requested)

    if allocated:
        print(f"Allocated {num_pages_requested} pages for {process_name}")
    else:
        print(f"Not enough memory available for {process_name} (requested
```

```
{num_pages_requested} pages)")

    memory_manager.print_memory_status()
    print()

memory_manager.print_total_memory_used()




# 6.2 ------------------------------------------------

class Page:
    def __init__(self, page_id, process_name=None):
        """Initialize a page with an ID and optional process name."""
        self.page_id = page_id
        self.process_name = process_name

    def allocate(self, process_name):
        """Allocate the page to a specific process."""
        self.process_name = process_name

    def deallocate(self):
        """Deallocate the page, making it available."""
        self.process_name = None

    def __str__(self):
        """String representation of the page status."""
        return f"Page {self.page_id}: {self.process_name or 'Free'}"


class MemoryManager:
    def __init__(self, num_pages, page_size):
        """Initialize the memory manager with pages and page size."""
        self.num_pages = num_pages
        self.page_size = page_size
        self.pages = [Page(page_id) for page_id in range(num_pages)]

    def allocate_memory(self, process_name, num_pages_requested):
        """Allocate pages to a process if enough pages are available."""
        free_pages = [page for page in self.pages if page.process_name is None]

        if len(free_pages) >= num_pages_requested:
            for i in range(num_pages_requested):
                free_pages[i].allocate(process_name)
            return True  # Allocation successful
        else:
            return False  # Not enough free pages

    def print_memory_status(self):
        """Print the status of all pages."""
        print("\nMemory Status:")
        for page in self.pages:
            print(page)

    def print_total_memory_used(self):
        """Print the total memory used in KB."""
        used_pages = sum(1 for page in self.pages if page.process_name is not None)
        total_memory_used = used_pages * self.page_size
        print(f"\nTotal memory used: {total_memory_used} KB\n")


# Driver Code
memory_manager = MemoryManager(num_pages=200, page_size=8)  # Page size = 8 KB

requests = [
```

```
    ("Student Records System", 40),
    ("Faculty Management System", 25),
    ("Library Information System", 30),
    ("Online Learning Platform", 35),
    ("Research Database", 50)
]

for request in requests:
    process_name, num_pages_requested = request
    allocated = memory_manager.allocate_memory(process_name, num_pages_requested)

    if allocated:
        print(f"Allocated {num_pages_requested} pages for {process_name}")
    else:
        print(f"Not enough memory available for {process_name} (requested
{num_pages_requested} pages)")

    memory_manager.print_memory_status()
    print()

memory_manager.print_total_memory_used()




# 6.3 ------------------------------------------------

class Page:
    def __init__(self, page_id, process_name=None):
        """Initialize a page with its ID and an optional process name."""
        self.page_id = page_id
        self.process_name = process_name

    def allocate(self, process_name):
        """Allocate the page to the specified process."""
        self.process_name = process_name

    def deallocate(self):
        """Deallocate the page, making it available for future use."""
        self.process_name = None

    def __str__(self):
        """Return a string representation of the page's status."""
        return f"Page {self.page_id}: {self.process_name or 'Free'}"


class MemoryManager:
    def __init__(self, num_pages, page_size):
        """Initialize the memory manager with the given pages and page size."""
        self.num_pages = num_pages
        self.page_size = page_size  # in KB
        self.pages = [Page(page_id) for page_id in range(num_pages)]

    def allocate_memory(self, process_name, num_pages_requested):
        """Allocate the requested number of pages to a process."""
        free_pages = [page for page in self.pages if page.process_name is None]

        if len(free_pages) >= num_pages_requested:
            # Allocate the requested pages to the process
            for i in range(num_pages_requested):
                free_pages[i].allocate(process_name)
            return True  # Allocation successful
        else:
            return False  # Not enough free pages

    def print_memory_status(self):
```

```
        """Print the status of each page in memory."""
        print("\nMemory Status:")
        for page in self.pages:
            print(page)

    def print_total_memory_used(self):
        """Print the total memory used in KB."""
        used_pages = sum(1 for page in self.pages if page.process_name is not None)
        total_memory_used = used_pages * self.page_size
        print(f"\nTotal Memory Used: {total_memory_used} KB\n")


# Driver Code
memory_manager = MemoryManager(num_pages=256, page_size=16)  # 256 pages, each 16 KB

requests = [
    ("Game Session 1", 32),
    ("Game Session 2", 20),
    ("Game Session 3", 40),
    ("Game Session 4", 18),
    ("Game Session 5", 25)
]

for request in requests:
    process_name, num_pages_requested = request
    allocated = memory_manager.allocate_memory(process_name, num_pages_requested)

    if allocated:
        print(f"Allocated {num_pages_requested} pages for {process_name}")
    else:
        print(f"Not enough memory available for {process_name} (requested
{num_pages_requested} pages)")

    memory_manager.print_memory_status()

memory_manager.print_total_memory_used()




# 6.4 ------------------------------------------------

class Page:
    def __init__(self, page_id, process_name=None):
        """Initialize a page with an ID and optional process_name."""
        self.page_id = page_id
        self.process_name = process_name

    def allocate(self, process_name):
        """Allocate the page to a given process."""
        self.process_name = process_name

    def deallocate(self):
        """Deallocate the page, making it available."""
        self.process_name = None

    def __str__(self):
        """Return a string representation of the page."""
        status = f"Page {self.page_id}: "
        if self.process_name:
            status += f"Allocated to {self.process_name}"
        else:
            status += "Free"
        return status
```

```python
class MemoryManager:
    def __init__(self, num_pages, page_size):
        """Initialize memory with a given number of pages and page size."""
        self.pages = [Page(page_id) for page_id in range(num_pages)]
        self.page_size = page_size  # Page size isn't actively used but could be useful for
extensions

    def allocate_memory(self, process_name, num_pages_requested):
        """Allocate memory to a process if enough free pages are available."""
        free_pages = [page for page in self.pages if page.process_name is None]

        if len(free_pages) >= num_pages_requested:
            # Allocate requested number of pages to the process
            for i in range(num_pages_requested):
                free_pages[i].allocate(process_name)
            return True
        else:
            return False

    def print_memory_status(self):
        """Print the status of all pages."""
        print("Memory Status:")
        for page in self.pages:
            print(page)

    def print_total_memory_used(self):
        """Print the total memory used by all processes."""
        used_pages = sum(1 for page in self.pages if page.process_name)
        print(f"Total Memory Used: {used_pages} pages")


# Driver Code
memory_manager = MemoryManager(num_pages=512, page_size=1)

requests = [
    ("Product Catalog Management", 80),
    ("Order Processing System", 120),
    ("Customer Database", 150),
    ("Payment Processing Gateway", 100),
    ("Inventory Tracking System", 60)
]

for process_name, num_pages_requested in requests:
    allocated = memory_manager.allocate_memory(process_name, num_pages_requested)
    if allocated:
        print(f"Allocated {num_pages_requested} pages for {process_name}")
        memory_manager.print_memory_status()
    else:
        print(f"Not enough memory available for {process_name} (requested
{num_pages_requested} pages)")

memory_manager.print_total_memory_used()



# 6.5 -----------------------------------------------

class Page:
    def __init__(self, page_id, process_name=None):
        self.page_id = page_id
        self.process_name = process_name

    def allocate(self, process_name):
        self.process_name = process_name

    def deallocate(self):
```

```
        self.process_name = None

    def __str__(self):
        return f"Page {self.page_id}: {self.process_name or 'Free'}"


class MemoryManager:
    def __init__(self, num_pages):
        self.pages = [Page(i) for i in range(num_pages)]
        self.total_memory_used = 0

    def allocate_memory(self, process_name, num_pages_requested):
        free_pages = [page for page in self.pages if page.process_name is None]
        if len(free_pages) < num_pages_requested:
            return False  # Not enough memory available

        for page in free_pages[:num_pages_requested]:
            page.allocate(process_name)
            self.total_memory_used += 1
        return True

    def print_memory_status(self):
        for page in self.pages:
            print(page)

    def print_total_memory_used(self):
        print(f"Total memory used: {self.total_memory_used} pages")


# Driver Code
memory_manager = MemoryManager(num_pages=512)
requests = [
    ("Product Catalog Management", 80),
    ("Order Processing System", 120),
    ("Customer Database", 150),
    ("Payment Processing Gateway", 100),
    ("Inventory Tracking System", 60)
]

for request in requests:
    process_name, num_pages_requested = request
    allocated = memory_manager.allocate_memory(process_name, num_pages_requested)
    if allocated:
        print(f"Allocated {num_pages_requested} pages for {process_name}")
    else:
        print(f"Not enough memory available for {process_name} (requested
{num_pages_requested} pages)")

memory_manager.print_total_memory_used()
```

```python
# 7.1

class Process:
    def __init__(self, pid, max_resources):
        self.pid = pid  # Process ID
        self.max_resources = max_resources  # Maximum resources the process can use
        self.allocated_resources = {resource: 0 for resource in max_resources}  # Currently
allocated resources

    def request_resource(self, resource, amount):
        """Request a specific amount of resource."""
        if resource not in self.max_resources:
            return False  # Resource not available for this process

        # Ensure the request doesn't exceed the maximum limit of the process
        if self.allocated_resources[resource] + amount > self.max_resources[resource]:
            return False

        self.allocated_resources[resource] += amount
        return True

    def release_resource(self, resource, amount):
        """Release a specific amount of resource."""
        if resource not in self.allocated_resources:
            return False  # Resource not allocated to this process

        # Ensure the process cannot release more than it currently holds
        if self.allocated_resources[resource] < amount:
            return False

        self.allocated_resources[resource] -= amount
        return True

    def __str__(self):
        """String representation of the process status."""
        return f"Process {self.pid} - Allocated: {self.allocated_resources}"


class ResourceManager:
    def __init__(self, resources):
        self.total_resources = resources  # Total available resources in the system
        self.available_resources = resources.copy()  # Track available resources
        self.processes = {}  # Store processes by their PID

    def add_process(self, process):
        """Add a process to the manager."""
        self.processes[process.pid] = process

    def request_resource(self, process_id, resource, amount):
        """Handle a resource request from a process."""
        if process_id not in self.processes:
            return False  # Process not found

        if resource not in self.available_resources:
            return False  # Resource not managed

        # Check if there are enough available resources
        if self.available_resources[resource] < amount:
            return False

        # Delegate the request to the process
        process = self.processes[process_id]
        if process.request_resource(resource, amount):
            self.available_resources[resource] -= amount  # Update available resources
            return True
        return False
```

```
    def release_resource(self, process_id, resource, amount):
        """Handle a resource release from a process."""
        if process_id not in self.processes:
            return False  # Process not found

        if resource not in self.available_resources:
            return False  # Resource not managed

        # Delegate the release to the process
        process = self.processes[process_id]
        if process.release_resource(resource, amount):
            self.available_resources[resource] += amount  # Update available resources
            return True
        return False

    def print_processes_status(self):
        """Print the status of all processes and available resources."""
        print("Available Resources:", self.available_resources)
        for process in self.processes.values():
            print(process)


# Driver Code
resources = {"CPU": 1, "Memory": 1024}
process1 = Process(1, {"CPU": 1, "Memory": 512})
process2 = Process(2, {"CPU": 1, "Memory": 768})

resource_manager = ResourceManager(resources)
resource_manager.add_process(process1)
resource_manager.add_process(process2)

print("Initial State:")
resource_manager.print_processes_status()

if resource_manager.request_resource(1, "CPU", 1):
    print("Process 1 allocated CPU")
else:
    print("Process 1 failed to allocate CPU")

if resource_manager.request_resource(1, "Memory", 512):
    print("Process 1 allocated Memory")
else:
    print("Process 1 failed to allocate Memory")

resource_manager.print_processes_status()

if resource_manager.request_resource(2, "Memory", 768):
    print("Process 2 allocated Memory")
else:
    print("Process 2 failed to allocate Memory")

resource_manager.print_processes_status()

if resource_manager.release_resource(1, "Memory", 512):
    print("Process 1 released Memory")
else:
    print("Process 1 failed to release Memory")

resource_manager.print_processes_status()




# 7.2 ------------------------------------------------

class Process:
```

```
    def __init__(self, pid):
        self.pid = pid  # Process ID
        self.dependencies = set()  # Set of dependent process PIDs

    def add_dependency(self, process):
        """Add a process as a dependency."""
        self.dependencies.add(process.pid)

    def remove_dependency(self, process):
        """Remove a process from dependencies."""
        self.dependencies.discard(process.pid)

    def __str__(self):
        """String representation of the process and its dependencies."""
        return f"Process {self.pid} -> Dependencies: {list(self.dependencies)}"


class WaitForGraph:
    def __init__(self):
        self.processes = {}  # Stores processes by their PID

    def add_process(self, process):
        """Add a process to the wait-for graph."""
        self.processes[process.pid] = process

    def add_dependency(self, process_pid, dependency_pid):
        """Add a dependency between two processes."""
        if process_pid in self.processes and dependency_pid in self.processes:
            self.processes[process_pid].add_dependency(self.processes[dependency_pid])

    def remove_dependency(self, process_pid, dependency_pid):
        """Remove a dependency between two processes."""
        if process_pid in self.processes and dependency_pid in self.processes:
            self.processes[process_pid].remove_dependency(self.processes[dependency_pid])

    def detect_deadlocks(self):
        """Detect cycles in the wait-for graph (indicating deadlocks)."""
        visited = set()  # Tracks visited nodes
        recursion_stack = set()  # Tracks nodes in the current path (cycle detection)

        # Check for cycles starting from each process
        for process in self.processes.values():
            if self.detect_cycle(process, visited, recursion_stack):
                return True  # Deadlock detected
        return False  # No deadlock detected

    def detect_cycle(self, process, visited, recursion_stack):
        """Detect cycle using DFS."""
        if process.pid in recursion_stack:
            return True  # Cycle detected

        if process.pid in visited:
            return False  # Already visited, no cycle from here

        # Mark the current process as visited and add to recursion stack
        visited.add(process.pid)
        recursion_stack.add(process.pid)

        # Recursively check all dependencies for cycles
        for dep_pid in process.dependencies:
            dep_process = self.processes[dep_pid]
            if self.detect_cycle(dep_process, visited, recursion_stack):
                return True

        # Backtrack: Remove from recursion stack
        recursion_stack.remove(process.pid)
        return False
```

```
    def print_wfg(self):
        """Print the current state of the wait-for graph."""
        print("Wait-For Graph:")
        for process in self.processes.values():
            print(process)


# Driver Code
process1 = Process(1)
process2 = Process(2)
process3 = Process(3)
process4 = Process(4)

wfg = WaitForGraph()
wfg.add_process(process1)
wfg.add_process(process2)
wfg.add_process(process3)
wfg.add_process(process4)

wfg.add_dependency(1, 2)
wfg.add_dependency(2, 3)
wfg.add_dependency(3, 4)
wfg.add_dependency(4, 1)  # This creates a cycle (deadlock)

wfg.print_wfg()

if wfg.detect_deadlocks():
    print("Deadlock detected in the system.")
    wfg.remove_dependency(4, 1)  # Resolving deadlock by breaking the cycle
    print("Deadlock resolved.")
else:
    print("No deadlock detected.")

wfg.print_wfg()
```

```
# 7.3 ------------------------------------------------


import threading
import time

# Resources available in the system
resources = {
    'computers': ['R1', 'R2', 'R3'],
    'projectors': ['P1', 'P2']
}

# Team requests for specific resources
team_requests = {
    'TeamA': {'computer': 'R1', 'projector': 'P1'},
    'TeamB': {'computer': 'R2', 'projector': 'P2'},
    'TeamC': {'computer': 'R3', 'projector': 'P2'}
}

# Track allocated resources for each team
allocated_resources = {
    'TeamA': {'computer': None, 'projector': None},
    'TeamB': {'computer': None, 'projector': None},
    'TeamC': {'computer': None, 'projector': None}
}
```

```
# Locks to control access to resources
resource_locks = {
    'computers': {res: threading.Lock() for res in resources['computers']},
    'projectors': {res: threading.Lock() for res in resources['projectors']}
}


def request_resource(team, resource_type, resource):
    """Request a resource for a team."""
    lock = resource_locks[resource_type][resource]
    acquired = lock.acquire(timeout=1)  # Try acquiring the resource with a timeout

    if acquired:
        allocated_resources[team][resource_type] = resource
        print(f"{team} acquired {resource_type} {resource}")
        return True
    else:
        print(f"{team} failed to acquire {resource_type} {resource}")
        return False


def release_resource(team, resource_type, resource):
    """Release a resource held by a team."""
    if resource is not None:
        resource_locks[resource_type][resource].release()
        allocated_resources[team][resource_type] = None
        print(f"{team} released {resource_type} {resource}")


def team_work(team):
    """Simulate the work of a team by acquiring and releasing resources."""
    # Get the resources requested by the team
    computer = team_requests[team]['computer']
    projector = team_requests[team]['projector']

    # Try to acquire both resources
    computer_acquired = request_resource(team, 'computers', computer)
    projector_acquired = request_resource(team, 'projectors', projector)

    if computer_acquired and projector_acquired:
        # Simulate work by sleeping
        print(f"{team} is working with {computer} and {projector}...")
        time.sleep(2)

        # Release the resources after work is done
        release_resource(team, 'computers', computer)
        release_resource(team, 'projectors', projector)
    else:
        # If any resource was not acquired, release the one that was acquired
        if computer_acquired:
            release_resource(team, 'computers', computer)
        if projector_acquired:
            release_resource(team, 'projectors', projector)

    print(f"{team} finished work.")


# Create and start threads for each team
threads = []
for team in team_requests.keys():
    thread = threading.Thread(target=team_work, args=(team,))
    threads.append(thread)
    thread.start()

# Wait for all threads to complete
for thread in threads:
    thread.join()
```

```python
print("Simulation complete.")


# 7.4 ------------------------------------------

import threading
import time
import random

class Philosopher(threading.Thread):
    def __init__(self, philosopher_id, left_utensil, right_utensil):
        super().__init__()
        self.philosopher_id = philosopher_id
        self.left_utensil = left_utensil
        self.right_utensil = right_utensil

    def run(self):
        while True:
            self.think()
            self.eat()

    def think(self):
        print(f"Philosopher {self.philosopher_id} is thinking.")
        time.sleep(random.uniform(1, 3))  # Simulate thinking time

    def eat(self):
        # Pick up utensils in a specific order to avoid deadlock
        if self.philosopher_id % 2 == 1:  # Odd philosophers pick up left first
            with self.left_utensil:
                with self.right_utensil:
                    print(f"Philosopher {self.philosopher_id} is eating.")
                    time.sleep(random.uniform(1, 2))  # Simulate eating time
        else:  # Even philosophers pick up right first
            with self.right_utensil:
                with self.left_utensil:
                    print(f"Philosopher {self.philosopher_id} is eating.")
                    time.sleep(random.uniform(1, 2))  # Simulate eating time

# Driver Code
num_philosophers = 5
utensils = [threading.Lock() for _ in range(num_philosophers)]
philosophers = []

for i in range(num_philosophers):
    left_utensil = utensils[i]
    right_utensil = utensils[(i + 1) % num_philosophers]
    philosopher = Philosopher(i + 1, left_utensil, right_utensil)
    philosophers.append(philosopher)

for philosopher in philosophers:
    philosopher.start()




# 7.5 ------------------------------------------

def is_safe_state(processes, available, allocation, need):
    work = available[:]
    finish = [False] * len(processes)
    safe_sequence = []

    while len(safe_sequence) < len(processes):
        allocated = False
        for i in range(len(processes)):
            if not finish[i] and all(need[i][j] <= work[j] for j in range(len(available))):
```

```
                work = [work[j] + allocation[i][j] for j in range(len(available))]
                finish[i] = True
                safe_sequence.append(processes[i])
                allocated = True
                break
        if not allocated:  # No process could be allocated
            return False, []

    return True, safe_sequence

# Driver Code
processes = ['P1', 'P2', 'P3', 'P4', 'P5']
available = [3, 2, 2]
allocation = [
    [1, 2, 2],
    [2, 0, 1],
    [1, 1, 1],
    [2, 1, 0],
    [0, 0, 2]
]
maximum = [
    [1, 2, 2],
    [2, 0, 1],
    [1, 1, 1],
    [2, 1, 0],
    [0, 0, 2]
]
need = [[maximum[i][j] - allocation[i][j] for j in range(len(available))] for i in
range(len(processes))]

is_safe, safe_sequence = is_safe_state(processes, available, allocation, need)

if is_safe:
    print("The system is in a safe state.")
    print("Safe sequence:", safe_sequence)
else:
    print("The system is in an unsafe state. Deadlock may occur.")
```

```python
#8.1
def fcfs_disk_scheduling(initial, requests):
    total = 0
    current = initial
    visited = [initial]
    for i in requests:
        total += abs(i - current)
        current = i
        visited.append(i)
    return total, visited
initial = 150
disk_requests = [200, 50, 800, 300, 100]
total,visited = fcfs_disk_scheduling(initial, disk_requests)
print("Total Head Movement:", total)
print("Order of Tracks Visited:", "â".join(map(str, visited)))

#8.2 ---------------------------------------------------------------------------
def sstf_disk_scheduling(initial, requests):
    total_head_movement = 0
    current = initial
    tracks_visited = [initial]
    remaining = requests.copy()
    while remaining:
        closest_track = min(remaining, key=lambda track: abs(track - current))
        movement = abs(closest_track - current)
        total_head_movement += movement
        current = closest_track
        tracks_visited.append(closest_track)
        remaining.remove(closest_track)
    return total_head_movement, tracks_visited
initial_position = 750
disk_requests = [1200, 500, 900, 1500, 300]
total_head_movement, tracks_visited = sstf_disk_scheduling(initial_position, disk_requests)
print("Total Head Movement:", total_head_movement)
print("Order of Tracks Visited:", tracks_visited)


#8.3 ---------------------------------------------------------------------------
def scan_disk_scheduling(initial, requests, direction='up', max=4999):
    total_head_movement = 0
    visited = []
    h_requests = sorted([track for track in requests if track >= initial])
    l_requests = sorted([track for track in requests if track < initial], reverse=True)
    if direction == 'up':
        for track in h_requests:
            total_head_movement += abs(track - initial)
            visited.append(track)
            initial = track
        total_head_movement += abs(max - initial)
        visited.append(max)
        initial = max
        for track in l_requests:
            total_head_movement += abs(track - initial)
            visited.append(track)
            initial = track
    elif direction == 'down':
        for track in l_requests:
            total_head_movement += abs(track - initial)
            visited.append(track)
            initial = track
        total_head_movement += abs(0 - initial)
        visited.append(0)
        initial = 0
        for track in h_requests:
            total_head_movement += abs(track - initial)
            visited.append(track)
            initial = track
```

```
    return total_head_movement, visited
initial_position = 2500
disk_requests = [2800, 1500, 3500, 4000, 1000]
total_head_movement, visited = scan_disk_scheduling(initial_position, disk_requests,
direction='up')
print("Total Head Movement:", total_head_movement)
print("Order of Tracks Visited:", visited)

#8.4 ---------------------------------------------------------------------------
def cscan_disk_scheduling(initial, requests, max=9999):
    total_head_movement = 0
    visited = []
    h_requests = sorted([track for track in requests if track >= initial])
    l_requests = sorted([track for track in requests if track < initial])
    for track in h_requests:
        total_head_movement += abs(track - initial)
        visited.append(track)
        initial = track
    total_head_movement += abs(max - initial)
    visited.append(max)
    initial = max
    total_head_movement += max
    initial = 0
    visited.append(0)
    for track in l_requests:
        total_head_movement += abs(track - initial)
        visited.append(track)
        initial = track
    return total_head_movement, visited
initial_position = 4000
disk_requests = [4200, 1000, 6000, 7500, 2000]
total_head_movement, visited = cscan_disk_scheduling(initial_position, disk_requests)
print("Total Head Movement:", total_head_movement)
print("Order of Tracks Visited:", visited)

#8.5----------------------------------------------------------------------------

def cscan_disk_scheduling(initial, requests, max=7999):
    total_head_movement = 0
    visited = []
    h_requests = sorted([track for track in requests if track >= initial])
    l_requests = sorted([track for track in requests if track < initial])
    for track in h_requests:
        total_head_movement += abs(track - initial)
        visited.append(track)
        initial = track
    total_head_movement += abs(max - initial)
    visited.append(max)
    initial = max
    total_head_movement += max
    initial = 0
    visited.append(0)
    for track in l_requests:
        total_head_movement += abs(track - initial)
        visited.append(track)
        initial = track
    return total_head_movement, visited
initial_position = 3500
disk_requests = [3800, 600, 7000, 1500, 2500]
total_head_movement, visited = cscan_disk_scheduling(initial_position, disk_requests)
print("Total Head Movement:", total_head_movement)
print("Order of Tracks Visited:", visited)
```

```python
# 9.1

import threading
import random

class BookInventory:
    def __init__(self):
        self.inventory = {'Book1': 3, 'Book2': 2, 'Book3': 1}
        self.lock = threading.Lock()

    def update_inventory(self, book, action):
        with self.lock:
            if action == 'checkout' and self.inventory.get(book, 0) > 0:
                self.inventory[book] -= 1
                print(f"{book} checked out. Remaining: {self.inventory[book]}")
            elif action == 'return':
                self.inventory[book] = self.inventory.get(book, 0) + 1
                print(f"{book} returned. Available: {self.inventory[book]}")
            else:
                print(f"{book} is out of stock.")

def user_request(inventory, book, action):
    inventory.update_inventory(book, action)

inventory = BookInventory()
threads = [threading.Thread(target=user_request, args=(inventory, random.choice(['Book1',
'Book2', 'Book3']), random.choice(['checkout', 'return']))) for _ in range(10)]

[t.start() for t in threads]
[t.join() for t in threads]

print("All operations completed.")




# 9.2

import threading
import time
import random

class ReservationSystem:
    def __init__(self, num_tables):
        self.tables = {f'Table{i+1}': True for i in range(num_tables)}
        self.lock = threading.Lock()

    def reserve_table(self, table_id):
        with self.lock:
            if self.tables.get(table_id, False):
                self.tables[table_id] = False
                print(f"{table_id} reserved.")
                return True
            print(f"{table_id} is already reserved.")
            return False

    def release_table(self, table_id):
        with self.lock:
            if table_id in self.tables and not self.tables[table_id]:
                self.tables[table_id] = True
                print(f"{table_id} released.")

def customer_request(system, table_id):
    if system.reserve_table(table_id):
        time.sleep(random.uniform(0.5, 2.0))
        system.release_table(table_id)
```

```
# Driver Code
system = ReservationSystem(3)
threads = [threading.Thread(target=customer_request, args=(system,
random.choice(list(system.tables.keys())))) for _ in range(20)]
[t.start() for t in threads]
[t.join() for t in threads]

print("All reservations completed.")




# 9.3

import threading
import time
import random

class InventorySystem:
    def __init__(self):
        self.inventory = {'item1': 100, 'item2': 100, 'item3': 100}
        self.lock = threading.Lock()

    def update_item(self, item, quantity):
        with self.lock:
            if self.inventory.get(item, 0) + quantity >= 0:
                self.inventory[item] += quantity
                print(f"{'Added' if quantity > 0 else 'Removed'} {abs(quantity)} of {item}.
Stock: {self.inventory[item]}")
                return True
            print(f"Failed to remove {abs(quantity)} of {item}. Insufficient stock.")
            return False

class ShoppingCart:
    def __init__(self):
        self.carts = {}
        self.lock = threading.Lock()

    def modify_cart(self, user_id, item, quantity):
        with self.lock:
            cart = self.carts.setdefault(user_id, {})
            cart[item] = cart.get(item, 0) + quantity
            if cart[item] <= 0:
                del cart[item]
            print(f"{user_id} {'added' if quantity > 0 else 'removed'} {abs(quantity)} of
{item} to their cart.")

def customer_action(user_id, cart, inventory):
    for _ in range(5):
        item = random.choice(list(inventory.inventory.keys()))
        quantity = random.randint(1, 5)
        action = random.choice(['add', 'remove'])

        if action == 'add' and inventory.update_item(item, -quantity):
            cart.modify_cart(user_id, item, quantity)
        elif action == 'remove' and cart.modify_cart(user_id, item, -quantity):
            inventory.update_item(item, quantity)

        time.sleep(random.uniform(0.1, 0.5))

# Driver Code
inventory = InventorySystem()
cart = ShoppingCart()
threads = [threading.Thread(target=customer_action, args=(f'User{i+1}', cart, inventory)) for
i in range(3) for _ in range(5)]

[t.start() for t in threads]
```

```
[t.join() for t in threads]

print("All customer actions completed.")




# 9.4

import threading
import time
import random

class Document:
    def __init__(self):
        self.lines = [f"Line {i+1}: Original text" for i in range(4)]
        self.lock = threading.Lock()

    def edit_line(self, line_number, new_text):
        with self.lock:
            if 0 <= line_number < len(self.lines):
                print(f"Editing line {line_number + 1}: '{self.lines[line_number]}' ->
'{new_text}'")
                self.lines[line_number] = new_text

    def show_document(self):
        print("\nCurrent Document Content:\n" + "\n".join(self.lines))

def user_edit(document, user_id):
    line_number = random.randint(0, len(document.lines) - 1)
    document.edit_line(line_number, f"Edited by {user_id} at {time.time()}")

# Driver Code
document = Document()
threads = [threading.Thread(target=user_edit, args=(document, f'User{i+1}')) for i in
range(2)]

[t.start() for t in threads]
[t.join() for t in threads]

document.show_document()
print("\nAll user edits completed.")




# 9.5

import threading
import random

class BankAccount:
    def __init__(self, balance):
        self.balance = balance
        self.lock = threading.Lock()

    def transaction(self, amount, is_deposit):
        with self.lock:
            if is_deposit:
                self.balance += amount
                print(f"Deposited ${amount}. Balance: ${self.balance}")
            elif self.balance >= amount:
                self.balance -= amount
                print(f"Withdrew ${amount}. Balance: ${self.balance}")
            else:
                print(f"Failed to withdraw ${amount}. Insufficient balance: ${self.balance}")
```

```
def perform_transaction(account):
    account.transaction(random.randint(50, 200), random.choice([True, False]))

# Driver Code
account = BankAccount(1000)
threads = [threading.Thread(target=perform_transaction, args=(account,)) for _ in range(3)]

[t.start() for t in threads]
[t.join() for t in threads]

print(f"\nFinal account balance: ${account.balance}")
```

```python
# 10.1

from collections import deque

class CafeFIFO:
    def __init__(self, capacity):
        # Initialize the queue with a maximum capacity
        self.capacity = capacity
        self.orders = deque()  # This will hold the orders

    def add_order(self, order):
        # Add a new order to the queue
        if len(self.orders) >= self.capacity:
            # If the queue is full, remove the oldest order (FIFO)
            removed_order = self.orders.popleft()
            print(f"Processed and removed order: {removed_order}")
        # Add the new order to the queue
        self.orders.append(order)
        print(f"Added order: {order}")

    def display_orders(self):
        # Display the current orders in the queue
        print("Current orders on the counter:")
        for order in self.orders:
            print(order)

# Driver Code
cafe = CafeFIFO(capacity=2)
orders = ["Coffee", "Muffin", "Croissant", "Smoothie"]
for order in orders:
    cafe.add_order(order)
    cafe.display_orders()



# 10.2

from collections import deque

class LibraryShelf:
    def __init__(self, capacity):
        # Initialize the shelf with a maximum capacity
        self.capacity = capacity
        self.books = deque()  # This will hold the books

    def add_book(self, book):
        # Add a new book to the shelf
        if len(self.books) >= self.capacity:
            # If the shelf is full, remove the oldest book (FIFO)
            removed_book = self.books.popleft()
            print(f"Removed book: {removed_book}")
        # Add the new book to the shelf
        self.books.append(book)
        print(f"Added book: {book}")

    def display_books(self):
        # Display the current books on the shelf
        print("Current books on the shelf:")
        for book in self.books:
            print(book)

# Driver Code
library_shelf = LibraryShelf(capacity=5)

books = [
    "The Great Adventure",
    "Mystery of the Lost City",
```

```
    "Journey Through Time",
    "Secrets of the Ocean",
    "The Final Frontier",
    "Legends of the Hidden Realm"
]

for book in books:
    library_shelf.add_book(book)
library_shelf.display_books()




# 10.3

from collections import OrderedDict

class CafeRecipeBook:
    def __init__(self, capacity):
        # Initialize the recipe book with a maximum capacity
        self.capacity = capacity
        self.recipes = OrderedDict()  # This will hold the recipes

    def add_recipe(self, recipe):
        # If the recipe is already in the counter, move it to the end (most recently used)
        if recipe in self.recipes:
            self.recipes.move_to_end(recipe)
        else:
            # If the counter is full, remove the least recently used recipe
            if len(self.recipes) >= self.capacity:
                removed_recipe = self.recipes.popitem(last=False)  # Remove the oldest item
                print(f"Removed recipe: {removed_recipe[0]}")
            # Add the new recipe to the counter
            self.recipes[recipe] = None  # We can use None as a placeholder value
            print(f"Added recipe: {recipe}")

    def display_recipes(self):
        # Display the current recipes on the counter
        print("Current recipes on the counter:")
        for recipe in self.recipes.keys():
            print(recipe)

# Driver Code
cafe_recipe_book = CafeRecipeBook(capacity=4)

recipes = [
    "Pumpkin Spice Latte",
    "Apple Pie",
    "Blueberry Muffin",
    "Cinnamon Roll",
    "Apple Pie",
    "Maple Pancakes",
    "Pumpkin Spice Latte"
]

for recipe in recipes:
    cafe_recipe_book.add_recipe(recipe)

cafe_recipe_book.display_recipes()




# 10.4

from collections import OrderedDict
```

```python
class ArtGalleryDisplay:
    def __init__(self, capacity):
        # Initialize the display with a maximum capacity
        self.capacity = capacity
        self.artworks = OrderedDict()  # This will hold the artworks

    def add_artwork(self, artwork):
        # If the artwork is already on the display, move it to the end (most recently used)
        if artwork in self.artworks:
            self.artworks.move_to_end(artwork)
        else:
            # If the display is full, remove the least recently used artwork
            if len(self.artworks) >= self.capacity:
                removed_artwork = self.artworks.popitem(last=False)  # Remove the oldest item
                print(f"Removed artwork: {removed_artwork[0]}")
            # Add the new artwork to the display
            self.artworks[artwork] = None  # We can use None as a placeholder value
            print(f"Added artwork: {artwork}")

    def display_artworks(self):
        # Display the current artworks on the board
        print("Current artworks on the display board:")
        for artwork in self.artworks.keys():
            print(artwork)

# Driver Code
art_gallery_display = ArtGalleryDisplay(capacity=3)

artworks = [
    "Sunset Over the Lake",
    "Abstract Dreams",
    "Cityscape at Dusk",
    "Golden Fields",
    "Mystic Mountains",
]

for artwork in artworks:
    art_gallery_display.add_artwork(artwork)

art_gallery_display.display_artworks()


# 10.5

from collections import defaultdict
import heapq

class LibraryDisplay:
    def __init__(self, capacity):
        self.capacity = capacity
        self.frequency = defaultdict(int)  # Track frequency of each book
        self.books = {}  # Store books and their frequencies
        self.min_heap = []  # Min-heap to track the least frequently used books

    def add_book(self, book, times_checked_out):
        # If the book is already in the display, update its frequency
        if book in self.books:
            self.frequency[book] += times_checked_out
        else:
            # If the display is full, remove the least frequently used book
            if len(self.books) >= self.capacity:
                # Pop the least frequently used book from the heap
                while self.min_heap:
                    least_frequent_book = heapq.heappop(self.min_heap)
                    if least_frequent_book in self.books:
                        del self.books[least_frequent_book]
                        break
```

```
            # Add the new book with its frequency
            self.books[book] = times_checked_out
            self.frequency[book] = times_checked_out

        # Push the book into the heap with its frequency
        heapq.heappush(self.min_heap, book)

    def display_books(self):
        # Sort books by frequency and print them
        sorted_books = sorted(self.books.items(), key=lambda x: (-self.frequency[x[0]], x[0]))
        for book, freq in sorted_books:
            print(f"{book} - {self.frequency[book]} times")

# Driver Code
library_display = LibraryDisplay(capacity=5)

book_checkouts = [
    ("The Great Gatsby", 3),
    ("1984", 5),
    ("To Kill a Mockingbird", 2),
    ("Pride and Prejudice", 4),
    ("The Catcher in the Rye", 1),
    ("Moby Dick", 6),
    ("The Odyssey", 3),
    ("War and Peace", 2)
]

for book, times_checked_out in book_checkouts:
    library_display.add_book(book, times_checked_out)

library_display.display_books()
```

```python
#11.1

import threading
import time
from queue import Queue

# Initialize a semaphore and a queue for bakers
oven_semaphore = threading.Semaphore(1)
request_queue = Queue()

def baker(name, bake_time):
    request_queue.put(name)  # Add the baker to the request queue

    while True:
        if request_queue.queue[0] == name:  # Check if this baker is at the front
            break
        time.sleep(0.1)  # Sleep briefly to avoid busy-waiting

    print(f"{name} is using the oven to bake.")
    time.sleep(bake_time)  # Simulate baking time
    print(f"{name} has finished baking.")

    request_queue.get()  # Remove the baker from the queue

# Driver Code
baker_threads = [threading.Thread(target=baker, args=(name, time)) for name, time in [("A",
2), ("B", 3), ("C", 1)]]

# Start and join all baker threads
for thread in baker_threads:
    thread.start()
for thread in baker_threads:
    thread.join()

print("All bakers have finished using the oven.")



#11.2

import threading
import time
from collections import deque

# Initialize semaphores for each coffee machine
machine_semaphores = {
    "Machine 1": threading.Semaphore(1),
    "Machine 2": threading.Semaphore(1)
}

# Queue to manage the order of baristas
request_queue = deque()
queue_lock = threading.Lock()

def barista(name, machine, preparation_time):
    with queue_lock:
        request_queue.append(name)

    while True:
        with queue_lock:
            if request_queue[0] == name:
                break
        time.sleep(0.1)

    machine_semaphores[machine].acquire()  # Acquire the appropriate machine
    print(f"{name} is using {machine} to prepare a drink.")
    time.sleep(preparation_time)
```

```
    print(f"{name} has finished preparing the drink.")
    machine_semaphores[machine].release()  # Release the machine

    with queue_lock:
        request_queue.popleft()  # Remove the barista from the queue

# Driver Code
barista_threads = [
    threading.Thread(target=barista, args=("Emma", "Machine 1", 2)),
    threading.Thread(target=barista, args=("Liam", "Machine 2", 3)),
    threading.Thread(target=barista, args=("Olivia", "Machine 1", 1))
]

# Start and join all barista threads
for thread in barista_threads:
    thread.start()
for thread in barista_threads:
    thread.join()

print("All baristas have finished using the coffee machines.")
```

```
# 11.3

import threading
import time
from collections import deque

# Initialize semaphore and request queue
conference_room_semaphore = threading.Semaphore(1)
request_queue = deque()
queue_lock = threading.Lock()

def employee(name, start_time, end_time):
    with queue_lock:
        request_queue.append(name)  # Add employee to the request queue

    while True:
        with queue_lock:
            if request_queue[0] == name:  # Check if this employee is at the front
                break
        time.sleep(0.1)  # Avoid busy-waiting

    conference_room_semaphore.acquire()  # Wait for the conference room
    print(f"{name} is using the conference room from {start_time} to {end_time}.")
    time.sleep(1)  # Simulate the time taken for the meeting
    print(f"{name} has finished using the conference room.")

    conference_room_semaphore.release()  # Release the conference room
    with queue_lock:
        request_queue.popleft()  # Remove employee from the queue

# Driver Code
employee_threads = [
    threading.Thread(target=employee, args=("A", "10:00", "11:00")),
    threading.Thread(target=employee, args=("B", "11:00", "12:00")),
]

# Start and join all employee threads
for thread in employee_threads:
    thread.start()
for thread in employee_threads:
    thread.join()
```

```
print("All employees have finished using the conference room.")



# 11.4

import threading
import time
from collections import deque

# Initialize semaphores for the stove and refrigerator
stove_semaphore = threading.Semaphore(1)
refrigerator_semaphore = threading.Semaphore(1)

# Queue to manage the order of cooks
request_queue = deque()
queue_lock = threading.Lock()

def cook(name, stove_time, refrigerator_time):
    with queue_lock:
        request_queue.append(name)  # Add cook to the request queue

    while True:
        with queue_lock:
            if request_queue[0] == name:  # Check if at the front of the queue
                break
        time.sleep(0.1)  # Avoid busy-waiting

    # Access the refrigerator first
    refrigerator_semaphore.acquire()
    print(f"{name} is using the refrigerator.")
    time.sleep(refrigerator_time)  # Simulate time to get ingredients
    print(f"{name} has finished using the refrigerator.")
    refrigerator_semaphore.release()

    # Then access the stove
    stove_semaphore.acquire()
    print(f"{name} is using the stove.")
    time.sleep(stove_time)  # Simulate cooking time
    print(f"{name} has finished using the stove.")
    stove_semaphore.release()

    with queue_lock:
        request_queue.popleft()  # Remove cook from the queue

# Driver Code
cook_threads = [
    threading.Thread(target=cook, args=("J", 2, 1)),
    threading.Thread(target=cook, args=("M", 3, 2)),
    threading.Thread(target=cook, args=("R", 1, 1))
]

# Start and join all cook threads
for thread in cook_threads:
    thread.start()
for thread in cook_threads:
    thread.join()

print("All cooks have finished using the stove and refrigerator.")



# 11.5
```

```python
import threading
import time
from collections import deque

# Initialize semaphore and request queue
water_pump_semaphore = threading.Semaphore(1)
request_queue = deque()
queue_lock = threading.Lock()

def gardener(name, water_time):
    with queue_lock:
        request_queue.append(name)  # Add gardener to the request queue

    while True:
        with queue_lock:
            if request_queue[0] == name:  # Check if at the front of the queue
                break
        time.sleep(0.1)  # Avoid busy-waiting

    water_pump_semaphore.acquire()  # Wait for the water pump
    print(f"{name} is using the water pump for {water_time} minutes.")
    time.sleep(water_time)  # Simulate watering time
    print(f"{name} has finished using the water pump.")

    water_pump_semaphore.release()  # Release the water pump
    with queue_lock:
        request_queue.popleft()  # Remove gardener from the queue

# Driver Code
gardener_threads = [
    threading.Thread(target=gardener, args=("Sophia", 30)),
    threading.Thread(target=gardener, args=("James", 45)),
    threading.Thread(target=gardener, args=("Oliver", 20))
]

# Start and join all gardener threads
for thread in gardener_threads:
    thread.start()
for thread in gardener_threads:
    thread.join()

print("All gardeners have finished using the water pump.")
```